

Aliza Zack Civil Engineering Web App — Structure & Purpose

Purpose

This webapp is a **civil engineering business site + portals** for Aliza Zack. It combines:

- A **marketing website** (services, projects, credentials, contact)
- A **“Request a Quote”** intake flow that stores submissions in **Firebase Firestore**
- A **client portal** (sign-in + role-gated dashboard)
- An **admin portal** (sign-in + role-gated dashboard)

The copy and README also reference roadmap items like **document sharing, project updates, and online payments**; the current Next.js app scaffolds these areas, while a legacy server contains stubbed payment endpoints.

What it's built with

- **Next.js 15 (App Router)** + **React 19** + **TypeScript**
- **Tailwind CSS** (utility-first styling)
- **Firebase client SDK**:
- **Auth** (email/password sign-in)
- **Firestore** (quotes + user roles)
- **Storage** (initialized; feature usage is a roadmap item)

Top-level structure

- `src/`
- `src/app/`: Next.js routes (App Router)
- `src/components/`: UI components, including the auth gate
- `src/lib/`: Firebase client setup + role lookup helpers
- `public/`: static assets for Next.js
- `legacy/`: older Node dev server + stub `/api/stripe/*` and `/api/paypal/*` endpoints
- `Config/tooling`: `package.json`, `next.config.ts`, `tsconfig.json`, `tailwind.config.ts`

Routes (App Router)

Public marketing pages

- / (home)
- /services
- /projects
- /credentials
- /contact
- /quote

Client portal

- /portal/sign-in
- /portal/dashboard

Admin portal

- /admin/sign-in
- /admin/dashboard

Authentication + roles (RBAC)

Authentication is handled via **Firestore Auth** (email/password).

Access control is implemented via a role gate:

- A user's role is fetched from Firestore: `users/{uid}.role`
- Expected roles are **client** and **admin**
- Client dashboard requires role **client**
- Admin dashboard requires role **admin**

Data layer (current)

There are **no Next.js API routes** in the modern app. Instead:

- The quote form writes directly to Firestore collection **quotes**
- Role checks read directly from Firestore documents in **users/{uid}**

Legacy server (payments placeholder)

The legacy/server/ directory contains a Node server and routes for:

- /api/stripe/* (stubbed)
- /api/paypal/* (stubbed)

These endpoints currently return “not implemented” responses and are not wired into the Next.js app.

“Build this app” prompt (implementation-matching spec)

Build a **Next.js (App Router) + TypeScript + Tailwind** webapp for a civil engineering business with:

- **Public marketing site** with pages Home, Services, Projects, Credentials, Contact, and Quote, all sharing a common header/footer and consistent layout container.
- **Quote request intake** at /quote that collects customer/project details and writes a Firestore document into quotes (include a server timestamp).
- **Firebase Auth** email/password sign-in for:
 - Client portal: /portal/sign-in
 - Admin portal: /admin/sign-in
- **Role-based access control**:
 - Store user roles in Firestore at users/{uid} with role: "client" | "admin".
 - Implement an AuthGate component that blocks/redirects unauthenticated users and enforces requireRole.
- **Dashboards**:
 - Client dashboard /portal/dashboard scaffolded for documents/updates/payments.
 - Admin dashboard /admin/dashboard scaffolded for quotes/projects management.
- **Configuration**:
 - Use NEXT_PUBLIC_FIREBASE_* environment variables to configure Firebase in the client.